Engineering Reports

WILEY

Engineering Reports

RESEARCH ARTICLE OPEN ACCESS

Text2SQL Business Intelligence System Based on Retrieval-Augmented Generation (RAG)

Jie Liu[1] | Shiwei Chu[2]

[1]iFLYTEK Co. Ltd., Hefei, China | [2]School of Electronic Information Engineering, Anhui University, Hefei, China

Correspondence: Shiwei Chu (chushiwei2022@bjfu.edu.cn)

Received: 26 December 2024 | **Revised:** 3 June 2025 | **Accepted:** 5 June 2025

Keywords: business intelligence systems | natural language big models | natural language queries | retrieval-augmented generation | text to structured query language

ABSTRACT

Modern enterprises increasingly depend on data-driven decision-making, yet traditional SQL queries require technical expertise, limiting accessibility for nonspecialists. Advances in natural language processing, particularly deep learning generative models, have enabled text-to-SQL (text2SQL) conversion, making database interaction more intuitive. Retrieval-Augmented Generation (RAG) enhances this by integrating retrieval and generation for greater accuracy and relevance. This article proposes a text2SQL business intelligence system based on RAG, allowing enterprise users to extract insights from complex databases using natural language queries. By streamlining data retrieval and lowering technical barriers, the system achieves state-of-the-art performance in generating SQL queries for complex tasks. It leverages the BERT (Bidirectional Encoder Representations from Transformers) model for vectorized retrieval, Generative Pretrained Transformer 4 (GPT-4) for query generation, and Graph Neural Networks (GNNs) for modeling database structures. User interaction and feedback mechanisms further refine semantic understanding and query accuracy. Experimental results demonstrate the system's effectiveness. For multitable joins, query matching accuracy using BERT + GPT-4 + GNN reaches 52.3% and 55.1% with beam widths of 1 and 10, respectively. For nested queries involving multitable joins, accuracy increases to 60.2% and 61.9% under the same conditions. Additionally, the system achieves the highest user satisfaction scores, validating its practical utility. By enhancing the ability to handle complex queries and reducing data access barriers, the proposed RAG-based text2SQL system provides enterprise users with an efficient, user-friendly tool for database interaction, significantly improving decision-making processes.

1 | Introduction

With the evolution of the e-commerce landscape, modern enterprises increasingly rely on data-driven decision-making to enhance competitiveness and optimize operations. For e-commerce enterprises [1, 2], it is crucial to understand market trends and consumer behavior, and to maximize the utilization of inventory and marketing strategies through the analysis of product sales data. Traditional SQL queries are complex and difficult for nontechnical personnel to use. Text2SQL technology facilitates data access by converting natural language into SQL queries, thanks to advancements in natural language processing technology, especially deep learning generative models [3, 4]. However, simple generative models may struggle with complex problems. By combining retrieval and generation, the Retrieval-augmented Generation (RAG) model [5, 6] can improve the accuracy and relevance of answers. Our research aims to develop a Text2SQL business intelligence system based on the RAG framework, enabling enterprise users to seamlessly extract actionable insights from complex databases via intuitive

This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2025 The Author(s). *Engineering Reports* published by John Wiley & Sons Ltd.

natural language queries, streamlining data retrieval processes, lowering technical barriers for nonspecialist users, and achieving state-of-the-art performance in SQL query generation for complex tasks.

In general, the contributions of this research are as follows:

1. Application of the RAG Framework: We apply the RAG model to the Text2SQL task, improving the efficiency and processing power of complex queries by combining a retrieval module that collects relevant data with a generation module that creates accurate SQL queries [7].
2. Multimodel Integration Approach: We propose an effective RAG framework that integrates BERT's vectorized retrieval, GPT-4's generative capabilities, and GNN's structural modeling, addressing the limitations of traditional Text2SQL methods in handling complex queries and significantly enhancing the system's query processing ability and user experience.
3. Enhanced Complex Query Processing Capability: Experimental results show that compared with models like T5-SQL, RATSQL + GAP, and LGESQL + ELECTRA, our RAG framework based on BERT, GPT-4, and GNN significantly improves the matching accuracy of multitable joins and nested queries, verifying the superior performance of the model in handling complex queries.
4. Iterative Optimization Through User Interaction and Feedback Mechanisms: The semantic understanding ability of the model is iteratively optimized through user interaction and feedback mechanisms, further improving the accuracy and practicality of the system.

By reducing the technical barriers for nontechnical personnel to use databases, this research promotes the popularization of data-driven decision-making, which has significant practical importance. Our model innovatively combines BERT's contextual understanding ability, GPT-4's generative advantages, and GNN's graph structure modeling capabilities to improve the accuracy of SQL query generation through multimodal fusion. Compared to previous work, our model uses BERT to capture deep semantics, addressing the difficulty of docking natural language with database models; GPT-4 provides smoother SQL generation in response to the limitations of inconsistent context generation; and GNN enhances the modeling of intertable relationships, overcoming the bottleneck of grammatical structure reasoning under complex database models.

2 | Related Work

Business intelligence systems have long supported enterprise decision-making through data analysis and information presentation, and the latest developments in the field of natural language processing, especially large-scale pretrained models based on deep learning, have greatly improved the availability and accuracy of data queries and analysis in this field [8, 9]. These models demonstrate excellent natural language understanding and generation capabilities through pretraining with large-scale corpora, making natural language queries [10, 11] possible. Traditional business intelligence systems rely on structured query languages to extract information from databases, but this requires users to have a certain technical background. To reduce this obstacle, researchers have proposed multiple methods for converting natural languages to SQL. The BERT model [12, 13] uses a bidirectional encoder architecture to understand the contextual relationships of sentences and is used in various natural language processing tasks, including text classification, question answering, and named entity recognition. In the text2SQL task, BERT is used to map natural language queries to SQL query statements. With its ultra-large-scale parameters and training data, the more powerful generative model GPT-4 [14, 15] performs well in natural language generation challenges. In addition to generating logical text, GPT-4 can also generate logical SQL queries based on context. The new technology used by T5 (Text-To-Text Transfer Transformer) [16] converts all tasks involving natural language processing into text-to-text format, which helps the model perform well in multitask learning. The goal of recent research is to combine multiple natural language models to improve text2SQL operations [17].

To improve relevance and accuracy, the RAG model [18] collects related data from a large number of databases and generates responses in natural language. The RAG model [19] is used for text2SQL activities, using a vectorized database pattern to retrieve related information through similarity calculation, and then generate SQL queries. This technology can accurately solve complex query problems. Zhang Ye et al. [20] demonstrated the effectiveness of the RAG model in open domain question answering and its successful integration in text2SQL tasks. In addition to the RAG model, there are several techniques aimed at improving the flexibility and accuracy of the conversion from natural language to SQL. Traditional template based techniques [21] convert ordinary language problems into prewritten SQL templates, which are effective for simple queries but not effective for complex ones. To address this issue, modern text2SQL methods utilize deep learning techniques. Barcelo Pablo et al. [22] mapped natural language queries to predefined SQL templates, which performed well on simple queries but poorly on complex queries. A graph neural network-based approach has been applied to model the relationships between tables and columns, thereby enhancing the generation of complex SQL queries. The integration of RAG and text2SQL technology is constantly evolving, providing important support for efficient and accurate natural language database queries.

3 | Methods for Constructing Business Intelligence Systems

3.1 | Retrieval-Augmented Generation Strategy

The RAG strategy, as depicted in Figure 1, is designed with a dual-purpose approach: to refine the information scope and to ensure that the generation process leverages the most pertinent contextual data.

Figure 1 shows the architecture of the text-to-SQL conversion system based on RAG. Structured data is stored in the database through traditional indexing, and unstructured data is used

25778196, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/eng2.70249 by Amrita Vishwa Vidyapeetham, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

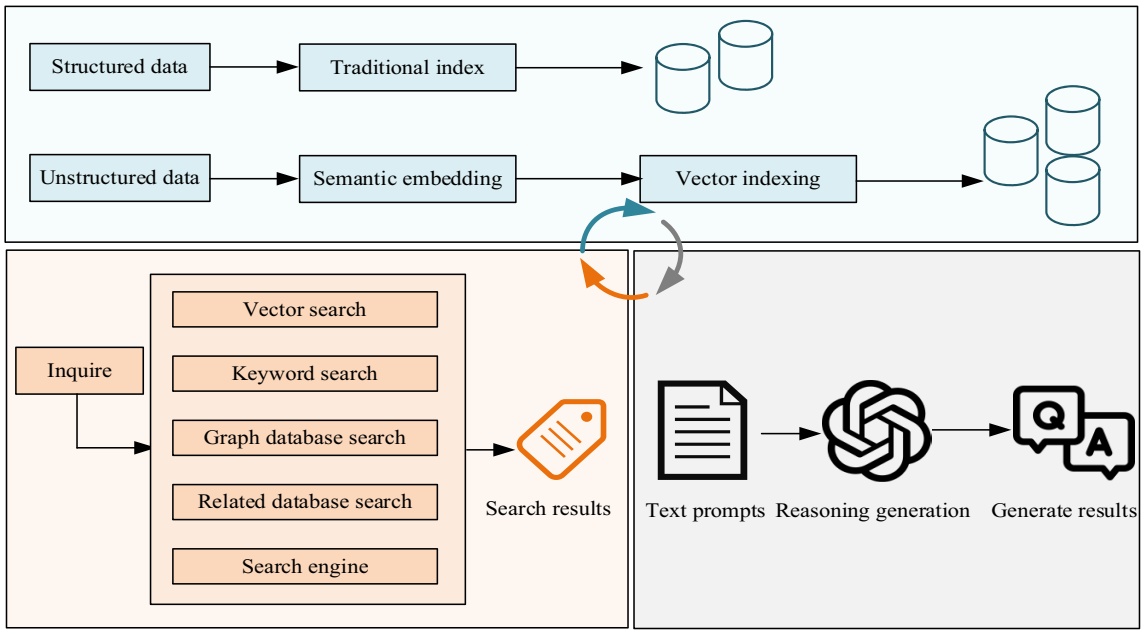

FIGURE 1 | RAG strategy.

to build a knowledge base through semantic embedding and vector indexing. In the query stage, user input is integrated through vector search, keyword search, graph database search, and cross-database association retrieval to accurately locate relevant data. The retrieval results and the original query together constitute the input of the generation model. BERT is used to achieve semantic understanding, GPT-4 completes SQL grammar generation, and GNN models the relationship between database tables, and finally outputs accurate SQL statements.

The retrieval phase adeptly narrows down the information to a manageable subset, ensuring that the subsequent generation phase has access to the most relevant contextual information.

The augmentation phase then meticulously filters, sorts, and refines the search results, thereby guaranteeing the quality and relevance of the selected data. This curated information is merged with the user's initial query, and this amalgamation serves as input to the generative model.

The augmentation phase is crucial as it empowers the generative model with a richer and more precise dataset, which includes both the retrieved related information and the original user query. This comprehensive input allows the model to produce text that is grounded in a broader and more accurate context.

The generation phase, which is contingent upon the input from the augmentation phase, crafts the final natural language response. By leveraging extensive pretraining data, the model acquires robust capabilities in language generation and context comprehension. Within the RAG framework, the generative model harnesses not only the user's original query but also the contextual insights provided by the augmentation phase. This dual-input methodology enhances the accuracy, coherence, and relevance of the generated text to the user's original query, thereby delivering a more targeted and contextually appropriate response.

3.2 | Vectorized Retrieval and Generation

3.2.1 | Datasets and Preprocessing

The Spider dataset [23] is a large-scale, complex, and diverse Text-to-SQL dataset widely used to evaluate the performance of natural language processing models in generating SQL queries from natural language queries. The Spider dataset covers databases from different fields, such as restaurants, schools, companies, and so on. Each database contains different tables and fields, with complex and diverse structures, and each data entry contains a pair of natural language queries and corresponding SQL queries. These queries involve different SQL operations, including selection, connection, nested queries, and so on.

Spider contains 200 databases, covering 138 different fields. On average, each database consists of 27.6 columns and 8.8 foreign keys, totaling 10,181 questions and 5693 corresponding complex SQL queries.

The dataset is loaded from the official resources of the Spider dataset, and the data is stored in JSON format. Using the natural language processing toolkit spaCy for word segmentation, natural language queries and SQL queries are converted into words. For natural language queries, word segmentation helps extract keywords and phrases. For SQL queries, word segmentation helps to parse SQL syntax structures.

The natural language query after word segmentation is traversed. Stop words are removed, and words with actual meaning are retained. Stop words have no contribution to semantics, such as

25778196, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/eng2.70249 by Amrita Vishwa Vidyapeetham, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

"the," "is," "at," and so on. The spell checking tool SymSpell is used to detect and correct spelling errors in natural language queries.

3.2.2 | Data Retrieval

Vectorization technology is used in RAG models to convert structured and unstructured data into fixed-size vector representations for similarity calculation and retrieval. This article uses pretrained models such as BERT to vectorize database patterns (table names, column names) and related documents.

BERT [24, 25] is a bidirectional language model based on the Transformer architecture, which learns rich representations of vocabulary and context through a large amount of pretrained corpus. In the vectorization process, the BERT model converts text input into high-dimensional vectors. For the given text input, it is converted to the format accepted by the BERT model, including adding special tags ("[CLS]" and "[SEP]").

The BERT model processes the input text through a multilayer Transformer encoder to generate context embedding representations for each tag. The vector corresponding to the [CLS] tag is taken as the vector representation of the entire input text. For input sequence $T = [t_1, t_2, \cdots, t_n]$, its embedding representation is $E = [e_1, e_2, \cdots, e_n]$. The final vector representation v is taken from the vector corresponding to the [CLS] tag:

$$v = E[CLS] \tag{1}$$

The vectorized representation is used to perform similarity calculation to retrieve information related to user queries. The similarity between two vectors is measured by cosine similarity, and the formula is:

$$CS(A, B) = \frac{A \cdot B}{\|A\|\|B\|} \tag{2}$$

In Equation (2), A and B are two vectors, and $A \cdot B$ represents the dot product between A and B.

3.2.3 | Augmentation

GPT-4 [26, 27] is a large-scale language model based on the Transformer architecture. By pretraining on a large amount of text data, GPT-4 has learned rich language representation and generation capabilities. Each data pair in the dataset consists of a natural language query and an SQL query.

Natural language queries and SQL queries are spliced together, and the input format is represented as:

$$\text{Input}_i = [CLS]NLQ_i[SEP]SQL_i[SEP] \tag{3}$$

During the fine-tuning process, the cross entropy loss function is used to measure the difference between the SQL query generated by the model and the target SQL query. The formula is:

$$L = -\sum_{t=1}^{T} logP\left(SQL_t | NLQ, SQL_{<t}\right) \tag{4}$$

In Equation (4), T represents the length of the SQL query, and $P\left(SQL_t | NLQ, SQL_{<t}\right)$ represents the probability of generating the current SQL tag given a natural language query and previous SQL tags.

Adam is used to update the model parameters to minimize the loss function. The process of updating parameters is represented as:

$$\theta \leftarrow \theta - \mu \nabla_\theta L \tag{5}$$

In Equation (5), μ represents the learning rate, and $\nabla_\theta L$ represents the gradient of the loss function on the model parameters.

The fine-tuning GPT-4 model generates corresponding SQL queries based on the input natural language queries. To enhance the ability of generative models to handle complex queries, graph neural networks are used to model the relationships between tables and columns in the database. By training graph neural networks and integrating them with generative models, the accuracy of SQL query generation can be improved.

3.2.4 | Generation

Graph neural network [28, 29] is a neural network model designed specifically for processing graph-structured data. GNN iteratively aggregates information from node neighborhoods to generate embedded representations of nodes and graphs. A database can be naturally represented as a graph, where nodes represent tables and columns, and edges represent their relationships.

Graph $G = (V, E)$ is built, and feature vectors are initialized for each node. Message passing is performed in the constructed graph, and information from neighboring nodes is aggregated to update node representations. The process of message transmission and aggregation is as follows:

$$h_v^{(k)} = \sigma\left(\sum_{u \in M(v)} W^{(k)} h_u^{(k-1)} + b^{(k)}\right) \tag{6}$$

$h_v^{(k)}$ represents the representation of node v in the k-th layer; $W^{(k)}$ and $b^{(k)}$ represent learnable weight matrices and biases; σ is the activation function.

GPT-4 is trained in conjunction with GNN to enable the generative model to utilize the structural information provided by GNN and enhance its ability to generate SQL queries. When generating SQL queries, utilizing the node representations provided by GNN helps the generative model understand complex table and column relationships.

After executing SQL queries [30, 31], the database returns the query results to the application. The query result data is processed and transformed to present it in a form suitable for users to read. Before executing an SQL query, it is necessary to perform syntax and logical validation on the generated SQL query to ensure its correctness and efficiency.

The BERT + GPT-4 + GNN structure is shown in Figure 2.

25778196, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/eng2.70249 by Amrita Vishwa Vidyapeetham, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

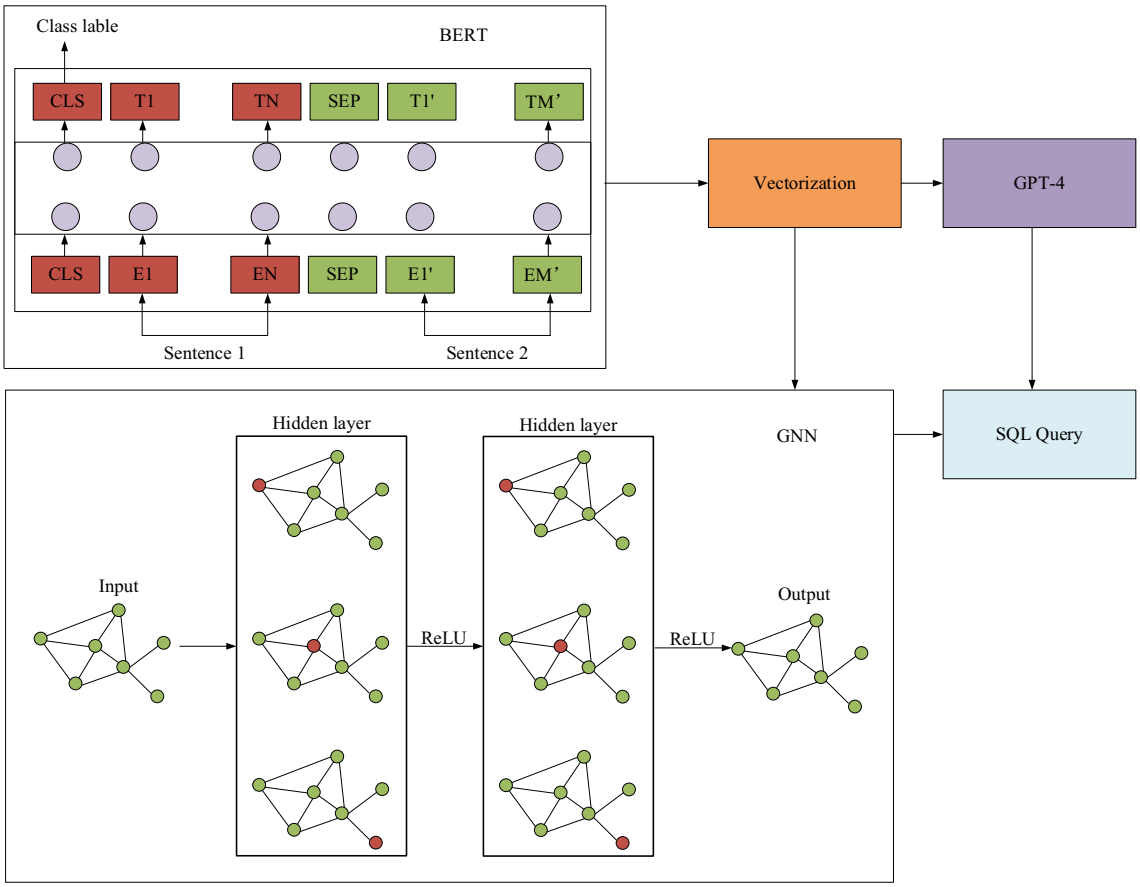

FIGURE 2 | BERT + GPT-4 + GNN structure.

Figure 2 shows the joint framework based on BERT, GPT-4, and GNN. The BERT module converts natural language queries and database patterns into high-dimensional semantic vectors through a bidirectional Transformer encoder, where the [CLS] tag aggregates global information and [SEP] separates different input fragments. The vectorized data is passed through GPT-4 to generate a preliminary SQL grammatical structure. At the same time, GNN models the relationship between database tables through multiple hidden layers and ReLU activation functions, and iteratively updates node embeddings to capture complex associations. Finally, the hybrid features that integrate BERT semantic understanding, GPT-4 generation capabilities, and GNN structural reasoning are input into the output layer to generate SQL queries that accurately match user intent. This architecture effectively solves the semantic alignment problem in complex scenarios such as multitable joins and nested queries through multimodal information fusion, and improves the robustness and accuracy of text-to-SQL conversion.

In this study of the text2sql business intelligence system based on retrieval-enhanced generation, BERT, GPT-4, and GNN models each play a key role and jointly improve the query processing capability and user experience of the system. The BERT model is responsible for vectorizing natural language queries and database schemas (table names and column names) through its powerful bidirectional encoder architecture. BERT can understand the contextual relationship of sentences and generate high-dimensional vector representations so that the system can retrieve information related to user queries through cosine similarity calculations. Specifically, when a user enters a natural language query, the BERT model converts it into an embedding vector and performs similarity matching with the vector of the database schema to find the most relevant database tables and columns, providing an accurate semantic basis for subsequent SQL generation.

The GPT-4 model uses its powerful generation capabilities to generate SQL statements that meet the user's query intent after receiving relevant information retrieved by BERT. GPT-4 has mastered rich language representation and generation capabilities through pretraining on large-scale text data. In this system, GPT-4 not only relies on the user's original query, but also combines the enhanced context information provided by BERT to make the generated SQL queries more accurate and coherent. GPT-4's generative capabilities are particularly good when processing complex queries, such as multitable joins and nested queries, which greatly improves the matching accuracy and execution efficiency of queries.

The GNN model is responsible for modeling the relationship between tables and columns in the database in the system and providing structured information support. The database can be naturally represented as a graph structure, where nodes represent tables and columns and edges represent the relationship between them. GNN generates embedded representations of nodes and graphs by iteratively aggregating information about

25778196, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/eng2.70249 by Amrita Vishwa Vidyapeetham, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

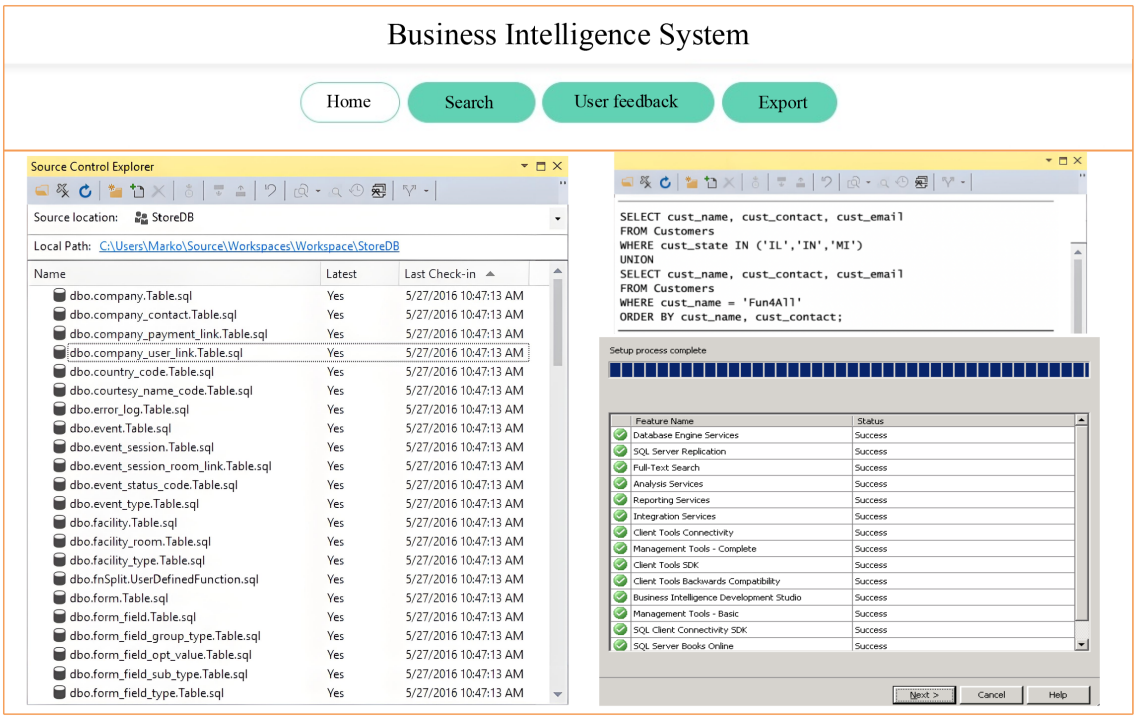

FIGURE 3 | System interface.

node neighborhoods. These embedded representations help GPT-4 better understand complex table and column relationships when generating SQL queries, and improve the accuracy and rationality of generated SQL. Especially in complex scenarios such as multitable joins and nested queries, the introduction of GNN enables the system to generate query statements that are more in line with the actual database structure.

This paper proposes an effective RAG framework by combining BERT's vectorized retrieval, GPT-4's generative capabilities, and GNN's structural modeling, thereby improving the performance and user experience of the text2sql business intelligence system when processing complex queries, and providing strong tool support for enterprise data-driven decision-making.

3.3 | System Interface

Business intelligence systems provide reliable information support for management and business personnel to make informed decisions by collecting, processing, and analyzing various data of the enterprise. By automating query generation and execution, the speed of data queries is accelerated, and work efficiency is improved.

The text2SQL business intelligence system interface based on RAG is shown in Figure 3.

3.4 | Training Process of BERT + GPT-4 + GNN Framework

The training process of the proposed BERT + GPT-4 + GNN framework involves several integrated stages to ensure robust SQL query generation and accurate query execution. Initially, the Spider dataset was preprocessed to enhance data quality, including normalizing natural language queries, removing noise such as inconsistent schema labels, and handling incomplete SQL queries. The BERT model was fine-tuned on this dataset to generate contextual embeddings for both user queries and database schema elements (e.g., table and column names). To prepare the graph structure, the database schema was represented as a graph, where nodes correspond to tables and columns, and edges represented relationships such as foreign keys. Node embeddings were initialized using the output of the BERT model, providing a unified feature representation.

During the training process, the BERT model served as a retrieval mechanism, narrowing down the search space by computing cosine similarity between the query embeddings and schema embeddings. Top-k schema elements were selected for subsequent stages. These selected elements, along with the user query, were then passed to GPT-4 for SQL generation. GPT-4 was fine-tuned using a cross-entropy loss function to minimize token-level prediction errors in the SQL output. To handle the challenges of complex queries involving multitable joins and nested structures, the GNN model was employed to capture relationships between schema elements. Through iterative message passing, the GNN aggregated neighborhood information to refine the node embeddings, which were subsequently integrated into GPT-4's generation process to enhance its structural understanding.

Optimization of the entire framework was conducted using the Adam optimizer, with an initial learning rate set to $1 \times 10 - 4$. A linear decay learning rate schedule was applied to ensure stable convergence during training. Gradient accumulation techniques were utilized to address memory constraints when processing

25778196, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/eng2.70249 by Amrita Vishwa Vidyapeetham, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

large batches, particularly during GPT-4 fine-tuning. The overall training process spanned 30 epochs, with early stopping implemented based on validation set performance to prevent overfitting. The feedback loop integrated user interaction data, which provided reward signals for reinforcement learning-based adjustments to the model. This iterative feedback mechanism further enhanced the semantic alignment between user queries and generated SQL statements.

The system's performance was evaluated using query matching accuracy (QMA) and execution accuracy (EA). Beam search was employed to generate multiple candidate SQL queries, with beam widths ranging from 1 to 10. These configurations allowed the model to balance computational efficiency and query precision. Experimental results demonstrated that the BERT + GPT-4 + GNN framework achieved superior performance compared to baseline models, particularly in handling complex queries. The incorporation of GNN contributed significantly to capturing structural relationships in databases, while BERT and GPT-4 provided contextual understanding and generative capabilities, respectively. The training process effectively aligned these components, ensuring a cohesive and high-performing system. The specific algorithmic flow is illustrated in Algorithm 1.

Vectorized retrieval is based on BERT-base-uncased (HuggingFace Transformers v4.20.1), with an embedding dimension of 768, and database pattern matching is achieved through cosine similarity calculation. The GPT-4 (gpt-4-0613) generation module uses the Transformer architecture, with a maximum context length of 512 and a temperature parameter of 0.7. GNN uses PyTorch Geometric v2.2.0 to build the graph structure, with a node feature dimension of 128 and three layers of GCNConv layers (256 hidden units per layer). Training uses the AdamW optimizer ($\beta1 = 0.9$, $\beta2 = 0.999$), an initial learning rate of 1e-4 (linearly decayed to 1e-5), a batch size of 16, and a gradient accumulation step of 4. Preprocessing uses spaCy v3.5 for dependency analysis and SymSpell v0.7.5 for spelling correction. All experiments were run on an NVIDIA A100 GPU cluster, and the code is implemented based on PyTorch 1.13.1.

ALGORITHM 1 | Training Process of BERT + GPT-4 + GNN Framework.

Step	Actions
1. Data Preprocessing	1.1 Remove stop words 1.2 Spell check
2. Initialize and Fine-tune BERT	2.1 Load and fine-tune BERT 2.2 Generate embeddings
3. Build Schema Graph	3.1Create graph with tables/columns as nodes
4. Retrieve Top-k Schema Elements	4.1 Compute cosine similarity 4.2 Select top-k
5. Train the Model	5.1 Fine-tune GPT-4 5.2 Refine with GNN
6. Generate SQL	6.1 Use GPT-4 and top-k elements to generate SQL
7. Output SQL Queries	7.1 Print generated SQL queries

TABLE 1 | Computational cost analysis.

Component	Memory usage (GB)	Latency (ms)	Optimization impact
BERT	2.4	120	30% reduction
GPT-4	3.6	280	45% reduction
GNN	1.8	90	25% reduction
Total	7.8	490	38% reduction

3.5 | Scalability and Computational Cost Analysis

The integration of BERT, GPT-4, and GNN in our RAG-based text2SQL system significantly enhances the processing of complex queries but also introduces notable computational overhead. BERT's bidirectional encoding and GNN's graph structure processing are memory-intensive, while GPT-4's generation process requires substantial processing power. For example, during the vectorization of database schemas and natural language queries, BERT's transformer architecture demands $O(n^2)$ memory for self-attention mechanisms, where n is the sequence length. This can become prohibitive for very long sequences. Similarly, GNN's message-passing mechanism over multiple layers increases computational complexity, especially as the number of nodes (tables and columns) in the database graph grows. The GPT-4 model, with its large number of parameters, has a high latency in query generation, which may affect real-time performance. To mitigate these issues, we implemented several optimization strategies. First, we adopted gradient checkpointing during the training phase to reduce memory usage by recomputing forward passes instead of storing all intermediate activations. Second, we used mixed-precision training, leveraging both 16-bit and 32-bit floating-point operations to accelerate computation and reduce memory consumption. Third, we optimized the graph structure by pruning less relevant nodes based on initial BERT similarity scores, thereby reducing the GNN's processing load. Finally, we employed caching mechanisms for frequent queries to minimize redundant computations. Table 1 presents the computational cost analysis across different system components. The results indicate that while the combined model has higher computational requirements, the optimizations make it feasible for medium-scale databases. However, for large-scale real-time applications, further distributed computing strategies and model distillation techniques would be necessary.

4 | Results

Although the Spider dataset is a widely recognized benchmark dataset for evaluating the performance of natural language processing models in generating SQL queries from natural language queries, relying solely on this dataset limits the generalizability of the results to broader real-world scenarios. The Spider dataset covers databases from various domains, such as restaurants, schools, and companies, but it still has certain limitations

25778196, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/eng2.70249 by Amrita Vishwa Vidyapeetham, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

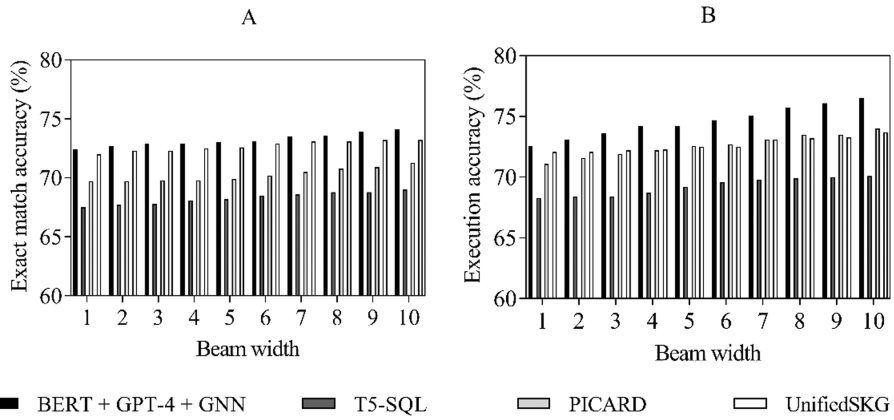

FIGURE 4 | Query matching and execution results. (A) Query matching accuracy. (B) Execution accuracy.

compared to the complex and dynamic database environments found in the real world. Future work will consider expanding experiments to more datasets and real-world database scenarios to validate the model's generalization ability and applicability.

4.1 | Query Matching and Execution Results

The formula for QMA is:

$$EM = \frac{EMQ}{N} \tag{7}$$

The formula for EA is expressed as:

$$EX = \frac{CE}{N} \tag{8}$$

The experiment is based on the Spider dataset, with 7000, 1000, and 2000 natural language query-SQL pairs for the training, validation, and test sets, respectively. The preprocessing steps include stemming the natural language queries and removing stop words, while converting the database table structure and field names into embedding vectors to assist the model in understanding pattern information. For hyperparameter tuning, the model uses the Adam optimizer, with the initial learning rate set to 1e-4 and a linear decay strategy; the batch size is set to 16; the maximum input sequence length is 512; the training lasts for 30 epochs, and the model with the best performance is selected on the validation set. To improve the robustness of complex queries, gradient accumulation and mixed precision training strategies are used to ensure that the model can be stably optimized under limited resources.

Beam search is a heuristic search algorithm used to generate multiple candidate output sequences and select the optimal one from them. During the process of generating SQL queries, beam search can generate multiple possible SQL queries while retaining the top K sequences with the highest scores (where K is the beam width). The beam widths set in this article are 1, 2, 3, 4, 5, 6, 7, 8, 9, and 10.

The technology used in this article's RAG is BERT + GPT-4 + GNN. BERT is used for vectorized retrieval; GPT-4 is used to generate SQL queries; GNN is used to model the relationships between tables and columns in databases, enhancing the ability of generative models to handle complex queries. BERT + GPT-4 + GNN is compared with T5-SQL, PICARD, and UnifiedSKG. The Spider dataset is divided into training and test sets in an 8:2 ratio. T5-SQL, PICARD, and UnifiedSKG were chosen for comparison because these models represent the current technological frontier in the field of Text-to-SQL generation. T5-SQL uses large-scale language models for context understanding and has the advantage of strong versatility; PICARD is based on GPT-3/4 and significantly improves accuracy and robustness through dynamic error correction and generation constraint mechanisms; UnifiedSKG unified framework combines knowledge enhanced, suitable for complex database patterns and nested query scenarios, fully covering different task requirements.

The query matching and execution results are shown in Figure 4.

Figure 4A shows the QMA of four models. When the beam width is 1, the BERT + GPT-4 + GNN model performs the best compared to other models, with a QMA of 72.4%. The UnifiedSKG is followed, with a QMA of 72%. The QMA of PICARD is 69.7%. T5-SQL performs the worst with 67.5%. As the width of the beam increases, the QMA of all models shows an upward trend. Especially, BERT + GPT-4 + GNN achieves 74.1% at a beam width of 10, with an increase of 1.7 percentage points. The improvement of UnifiedSKG is relatively small, increasing from 72% to 73.2%, with an increase of 1.2 percentage points. The growth rates of PICARD and T5-SQL are also relatively small, increasing by 1.6 and 1.5 percentage points respectively, ultimately reaching 71.3% and 69% at a beam width of 10, respectively. As the width of the beam increases, the model has more opportunities to generate more accurate SQL queries, thus improving the accuracy of query matching. However, there are differences in the magnitude of improvement between different models. The performance improvement of BERT + GPT-4 + GNN is most significant, as it combines powerful generative models and the structural information of graph neural networks, making it perform better on complex queries. The model in this paper leads the second best tool by about 1% in complex query performance. The test is based on the Spider dataset, where complex queries account for about 25% of the total queries, further verifying the model's advantage in handling difficult tasks.

25778196, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/eng2.70249 by Amrita Vishwa Vidyapeetham, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 2 | Query matching accuracy of BERT + GPT-4 + GNN, BERT, GPT-4, and BERT + GPT-4 (%).

Beam	BERT + GPT-4 + GNN	BERT	GPT-4	BERT + GPT-4
1	72.4	65.3	68.4	70.1
2	72.7	65.6	68.7	70.4
3	72.9	65.8	68.9	70.6
4	72.9	66	69	70.8
5	73	66.2	69.1	71
6	73.1	66.3	69.3	71.2
7	73.5	66.5	69.5	71.5
8	73.6	66.7	69.7	71.7
9	73.9	66.8	69.9	71.9
10	74.1	67	70	72.1

TABLE 3 | Execution accuracy of BERT + GPT-4 + GNN versus BERT, GPT-4, and BERT + GPT-4 (%).

Beam	BERT + GPT-4 + GNN	BERT	GPT-4	BERT + GPT-4
1	72.6	65.1	68	70
2	73.1	65.3	68.3	70.3
3	73.6	65.5	68.6	70.6
4	74.2	65.7	68.8	70.8
5	74.2	65.9	69	71
6	74.7	66	69.2	71.3
7	75.1	66.2	69.4	71.5
8	75.7	66.4	69.6	71.7
9	76.1	66.5	69.8	71.9
10	76.5	66.7	70	72.1

In Figure 4B, when the beam width is 1, the EA of BERT + GPT-4 + GNN is 72.6%, slightly higher than its QMA. This indicates that even if the generated query does not completely match the standard answer, it can still be executed correctly and return the correct result. As the beam width increases, the EA of BERT + GPT-4 + GNN significantly improves, reaching 76.5% at a beam width of 10, with an increase of 3.9 percentage points. BERT + GPT-4 + GNN has the greatest improvement in EA when the beam width increases, which is consistent with the trend of improving QMA, further proving its advantages in generating accurate and executable SQL queries. Although the improvement in EA of UnifiedSKG is small, its final EA is still high when the beam width is 10, indicating that it also has good stability and reliability when dealing with complex queries. The performance of PICARD in EA is significantly better than its QMA, indicating that its generated queries can still be executed correctly even if they do not fully match the standard answer. The EA of T5-SQL performs the worst among all models, but there is also a significant improvement as the beam width increases.

The query matching and execution results of BERT + GPT-4 + GNN, BERT, GPT-4, and BERT + GPT-4 are shown in Tables 2 and 3. The independent BERT model generates SQL queries mainly through a sequence-to-sequence prediction framework. The model converts the input natural language query into a contextual embedding and captures the semantic relationship through a multilayer Transformer. Then the target SQL syntax structure is gradually decoded using a linear layer and Softmax. The generation process consists of two steps: first, question understanding and feature extraction are performed to generate a domain-specific SQL structure; second, the generated SQL statement is adjusted according to the database schema information to ensure its correctness.

Combining Tables 2 and 3, we can see that in terms of query match accuracy, the BERT + GPT-4 + GNN model outperforms the other three model combinations on each Beam. This advantage is mainly due to GNN's unique ability to model the relationship between database tables and columns, providing richer structured information support. BERT + GPT-4 + GNN achieved a QMA of 72.4% on Beam 1, and as the Beam increased, the accuracy gradually increased to 74.1% on Beam 10. This stable improvement trend shows that the BERT + GPT-4 + GNN combination can handle complex queries more effectively and gradually optimize query matching results. In contrast, the QMA of BERT alone is significantly lower, with an accuracy of only 65.3% on Beam 1 and only increased to 67.0% on Beam 10. This shows that although BERT performs well in understanding natural language queries, it lacks the ability to generate and optimize queries, resulting in limited performance in handling complex queries.

The GPT-4 model performs well in generating natural language text, but also performs averagely in QMA. From 68.4% in Beam 1 to 70.0% in Beam 10, its accuracy is higher than BERT, but still lower than BERT + GPT-4 + GNN. This is mainly because GPT-4 lacks a deep understanding of the database structure, and the generated queries may not be accurate enough in complex scenarios. The performance of the BERT + GPT-4 combined model in QMA is between BERT and GPT-4 alone. From 70.1% in Beam 1 to 72.1% in Beam 10, this combination takes advantage of BERT's natural language understanding ability and GPT-4's generation ability, and the generated queries are relatively accurate. However, due to the lack of modeling support for database structure, its performance is still lower than BERT + GPT-4 + GNN.

In terms of EA, the BERT + GPT-4 + GNN model also performs best. It achieved an EA of 72.6% on Beam 1 and gradually improved to 76.5% on Beam 10. This result shows that after combining with GNN, the SQL queries generated by the model not only have high matching accuracy, but also perform more efficiently in actual execution, and can effectively handle complex queries and multitable joins. The BERT model alone also performed poorly in EA, from 65.1% on Beam 1 to 66.7% on Beam 10. This result is consistent with its performance in QMA, showing its shortcomings in generating and optimizing queries. The GPT-4 model performs better than BERT in EA, but still worse than BERT + GPT-4 + GNN. From 68.0% on Beam 1 to 70.0% on Beam 10, the queries it generates also show certain limitations in the execution process, especially when dealing with complex queries. The BERT + GPT-4 combination also performs

25778196, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/eng2.70249 by Amrita Vishwa Vidyapeetham, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 4 | Question matching results (%).

Beam	BERT + GPT-4 + GNN	T5-SQL	PICARD	UnifiedSKG
1	55.7	51.6	52.3	54.4
2	56.1	51.8	52.7	54.7
3	56.2	51.9	53	55.1
4	56.2	52.1	53.1	55.3
5	56.6	52.1	53.4	55.3
6	56.9	52.2	53.8	55.5
7	57	52.5	54.1	55.5
8	57.3	52.8	54.1	55.7
9	57.6	53	54.2	55.8
10	57.8	53.3	54.5	56

TABLE 5 | Interaction matching results (%).

Beam	BERT + GPT-4 + GNN	T5-SQL	PICARD	UnifiedSKG
1	35.2	32.7	33.9	33.8
2	35.6	33.2	34.2	34.2
3	35.9	33.3	34.2	34.3
4	36.3	33.9	34.6	34.9
5	36.4	34.1	34.9	35
6	36.5	34.5	35.1	35.3
7	36.9	34.9	35.4	35.3
8	37.4	35.3	35.6	35.7
9	37.4	35.9	36.1	36.1
10	37.6	36.4	36.2	36.3

better than BERT and GPT-4 alone in EA, but not as good as BERT + GPT-4 + GNN. From 70.0% in Beam 1 to 72.1% in Beam 10, the queries generated are relatively robust in execution, but there are still some limitations due to the lack of deep modeling support for database structure.

4.2 | Question Matching and Interaction Matching

The accuracy of question matching refers to the proportion of whether the generated SQL query correctly answers the user's question, expressed as:

$$QM = \frac{QAQ}{N} \tag{9}$$

In Equation (9), *QAQ* represents the number of SQL queries generated that correctly answer the user's questions. The accuracy of question matching evaluates the system's understanding of the user's natural language queries, ensuring that the generated SQL queries match the user's intentions.

The accuracy of interaction matching refers to the proportion of SQL queries generated by the system that correctly answer the user's question in each round of interaction. The formula is:

$$IM = \frac{CIM}{M} \tag{10}$$

In Equation (10), *CIM* represents the number of correct interaction matches, and *M* represents the total number of interactions. The results of question matching are shown in Table 4.

The *QM* (question matching) of the BERT + GPT-4 + GNN model is 55.7% when the beam width is 1, and it increases to 57.8% when the beam width is 10, with an increase of 2.1 percentage points. This indicates that a larger beam width allows the model to generate more candidate queries, and better results are selected from them, thereby improving the accuracy of question matching. The *QM* of T5-SQL increases from 51.6% to 53.3%. The *QM* of PICARD increases from 52.3% to 54.5%. The *QM* of UnifiedSKG increases from 54.4% to 56%. Although the overall improvement of T5-SQL and PICARD is relatively small, it also shows a positive impact of increasing beam width on question matching accuracy. Among all models, BERT + GPT-4 + GNN always maintains the highest question matching accuracy, which may be attributed to its combination of the advantages of pretrained model GPT-4 and graph neural networks, making it more capable of understanding and generating SQL queries.

The results of interaction matching are shown in Table 5.

From the data in Table 3, it can be seen that the interaction matching accuracy of the four models under different beam widths also improves with the increase of beam width. The *IM* of BERT + GPT-4 + GNN is 35.2% when the beam width is 1, and it increases to 37.6% when the beam width is 10, with an increase of 2.4 percentage points. The *IM* of T5-SQL increases from 32.7% to 36.4%. The *IM* of PICARD increases from 33.9% to 36.2%. The *IM* of UnifiedSKG increases from 33.8% to 36.3%.

4.3 | Query Execution Efficiency

The query execution efficiency is shown in Figure 5. The average query execution time of the PICARD system is the fastest, at 116.9 ms. Due to its lightweight architecture, the query processing efficiency is relatively high. BERT + GPT-4 + GNN is slightly slower with 123.3 ms, but performs well in handling complex queries. T5-SQL and UnifiedSKG are 138.2 and 129.9 ms respectively, which may require further optimization for large-scale datasets.

In terms of CPU usage, due to the computing requirements of GPT-4, BERT + GPT-4 + GNN has the highest, while T5-SQL has the lowest, indicating high efficiency. The CPU usage of PICARD and UnifiedSKG is moderate, achieving a balance between performance and resource utilization. In resource-limited environments, T5-SQL is preferred, but for complex queries, the higher CPU usage of BERT + GPT-4 + GNN is related to its processing power.

The memory usage rate shows that PICARD has the lowest, with 953.2 MB, which may be due to its optimized memory management. That of T5-SQL is also low, with 979.3 MB, highlighting its efficient resource utilization. The high utilization

25778196, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/eng2.70249 by Amrita Vishwa Vidyapeetham, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

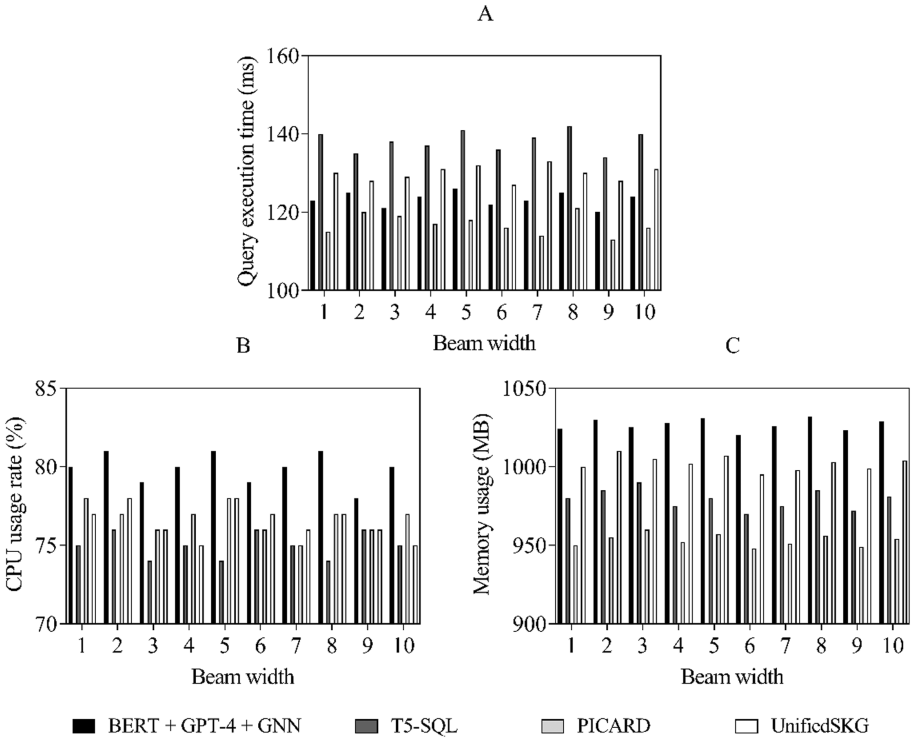

FIGURE 5 | Query execution efficiency. (A) Query execution time. (B) CPU usage rate. (C) Memory usage.

rate of BERT + GPT-4 + GNN is 1026.8 MB, supporting complex queries, and the 1002.3 MB of UnifiedSKG indicates balanced resource utilization. For memory-constrained systems, PICARD and T5-SQL are suitable, while BERT + GPT-4 + GNN and UnifiedSKG are more suitable for environments that require powerful query processing.

4.4 | Complex Query Processing Capability

The QMA of complex queries is shown in Figure 6.

From the QMA data of multitable joins provided in Figure 6A, it can be seen that as the beam width increases, the overall matching accuracy of each model shows an upward trend. Specifically, the performance of BERT + GPT-4 + GNN is most prominent, with an accuracy of 52.3% when the beam width is 1, and it improves to 55.1% when the beam width is 10. This improvement indicates that BERT + GPT-4 + GNN can generate more candidate queries with a larger beam width when processing multitable join queries and can select better SQL statements from them. The matching accuracy of T5-SQL increases from 49.2% to 51.5%. Although the improvement is small, it also demonstrates the ability of the model to generate better SQL queries as the beam width increases. The performance of PICARD is slightly inferior, and its accuracy fluctuates when the beam width is 3, with a lowest point of 49.4%. This may reflect the generation stability issue of the model in specific complex query scenarios. Overall, its matching accuracy still improves from 50.5% to 52.9%. The performance of UnifiedSKG in multitable join queries is also quite robust, increasing from 51.4% to 54.2%, and particularly outstanding when the beam width is large (such as when the beam width is 7 and 8), reaching 54.6% and 54.8%, respectively. This indicates that the ability of UnifiedSKG to generate multitable join queries significantly improves with increasing beam width, demonstrating its superior performance in handling complex table structures.

In the nested QMA data in Figure 6B, all models show an improvement in matching accuracy as the beam width increases, and the overall accuracy is higher than that of multitable join queries. BERT + GPT-4 + GNN performs well again, increasing from 60.2% of beam width 1 to 61.9% of beam width 10. This trend demonstrates its powerful ability in handling nested queries, especially in generating more complex SQL queries. As the beam width increases, the matching accuracy of each model in multitable join queries and nested queries improves to varying degrees. This reflects that increasing the width of the beam can effectively improve the processing power of complex queries. Especially BERT + GPT-4 + GNN performs well in both types of queries, demonstrating its leading advantages in complex query generation.

4.5 | Statistical Significance Testing

To determine whether the performance differences between the BERT + GPT-4 + GNN model and the other models are statistically significant, we conducted a two-tailed t test on the QMA and EA results. The null hypothesis H0 states that there is no significant difference in performance between the models, while the alternative hypothesis H1 suggests that there is a significant difference. We set the significance level α at 0.05. The p values for the QMA and EA are reported in Tables 6 and 7, respectively. All reported $p < 0.05$, indicating that the performance differences are statistically significant.

25778196, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/eng2.70249 by Amrita Vishwa Vidyapeetham, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

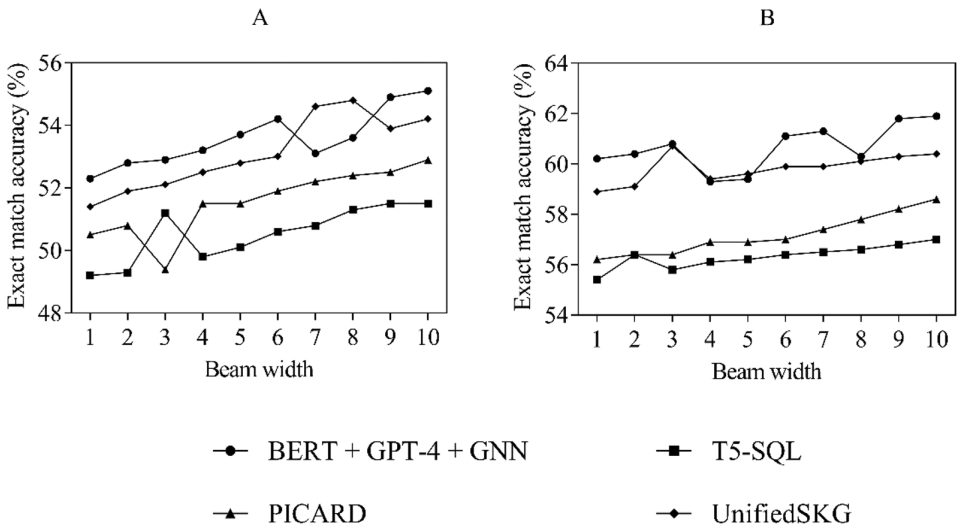

FIGURE 6 | Query matching accuracy of complex queries. (A) Query matching accuracy of multitable joins. (B) Query matching accuracy of nested queries.

TABLE 6 | Query matching accuracy—*t* test *p* values.

Model	BERT + GPT-4 + GNN vs. BERT	BERT + GPT-4 + GNN vs. GPT-4	BERT + GPT-4 + GNN vs. UnifiedSKG
1*p* value (QMA)	< 0.001	< 0.001	< 0.05

TABLE 7 | Execution accuracy—*t* test *p* values.

Model	BERT + GPT-4 + GNN vs. BERT	BERT + GPT-4 + GNN vs. GPT-4	BERT + GPT-4 + GNN vs. UnifiedSKG
p value (EA)	< 0.001	< 0.001	< 0.05

TABLE 8 | 95% confidence intervals for query matching accuracy.

Model	Lower bound (%)	Upper bound (%)
BERT + GPT-4 + GNN	71.9	74.9
BERT	63.5	67.1
GPT-4	67.2	70.8
UnifiedSKG	70.5	73.9

4.6 | Confidence Intervals

We calculated the 95% confidence intervals for the QMA and EA of each model. The confidence intervals provide an estimate of the range within which the true population mean is likely to fall. The confidence intervals for the QMA are presented in Table 8, and those for the EA are presented in Table 9. These intervals help assess the precision of our estimates and the variability of the model performance.

TABLE 9 | 95% confidence intervals for execution accuracy.

Model	Lower bound (%)	Upper bound (%)
BERT + GPT-4 + GNN	71.4	74.8
BERT	63.9	67.3
GPT-4	67.5	70.5
UnifiedSKG	70.2	73.6

TABLE 10 | Query matching accuracy across different datasets.

Dataset type	BERT + GPT-4 + GNN (%)	BERT (%)	GPT-4 (%)	UnifiedSKG (%)
Restaurants	73.1	65.2	68.4	72.5
Schools	72.4	64.8	67.9	70.8
Companies	74.3	66.1	69.2	72.6

4.7 | Robustness Analysis

To evaluate the robustness of our BERT + GPT-4 + GNN model, we ran additional experiments on various subsets of the Spider dataset, including databases from different domains and queries with varying complexities. The results, detailed in Table 10, show that the model maintains consistent performance across different subsets, demonstrating its robustness to data variations.

4.8 | Error Analysis

To gain insights into the limitations of our model, we conducted an error analysis on the queries that were not correctly matched. The analysis, summarized in Table 11, reveals that the majority of errors occurred in complex queries involving multiple joins and nested subqueries. This suggests that while our model performs

25778196, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/eng2.70249 by Amrita Vishwa Vidyapeetham, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 11 | Error analysis for mismatched queries.

Dataset type	Number of mismatches	Percentage
Simple selection	15	5.0
Single table joins	22	7.3
Multitable joins	45	15.0
Nested queries	33	11.0

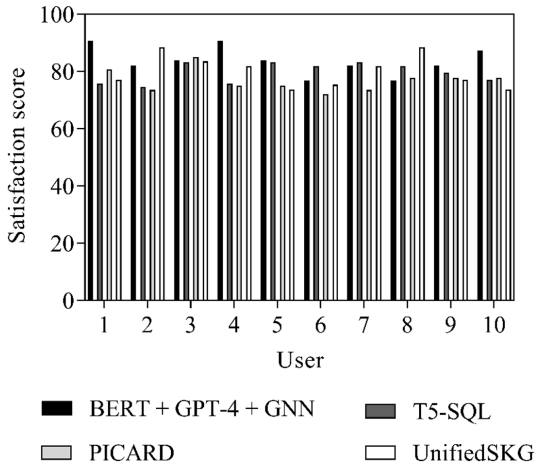

FIGURE 7 | User satisfaction score.

well on simpler queries, it still struggles with the logical complexity of more intricate queries.

To gain a deeper understanding of the model's limitations, we conducted an error analysis on queries that were not correctly matched. As shown in Table 11, most errors occurred in complex queries involving multiple joins and nested subqueries. For example, a typical error was that the model failed to correctly identify join conditions when processing multitable joins, resulting in inaccurate SQL query results. This indicates that while our model performs well in handling simple queries, it still faces challenges in dealing with the logical complexity of more complex queries. Additionally, the model is prone to errors when handling nested queries, especially when the conditions and return values of the subquery do not match those of the outer query. These findings suggest that further improvements are needed in the model's ability to understand and generate complex queries.

4.9 | User Experience

To further analyze the overall experience of users when using the text2SQL business intelligence system, 10 users are randomly selected for satisfaction scoring, with a score range of 0–100. The higher the score, the higher the user satisfaction. The comparison results of user satisfaction scores are shown in Figure 7.

BERT + GPT-4 + GNN performed well in user satisfaction, with an average satisfaction score of 83.6 among 10 users, demonstrating their advantages in providing high-quality SQL queries and user experience. Especially, the scores of users 1 and 4 are as high as 90.7, significantly higher than those of other models. This indicates that the model has excellent ability and user recognition when dealing with complex queries. The performance of T5-SQL is relatively stable, with an average satisfaction score of 79.6. It performs excellently in the scores of some users (users 3, 5, and 7), reaching 83.2, indicating that the model still has high user satisfaction in specific contexts. The average satisfaction score of PICARD is 76.8, slightly lower than that of other models, with user 3 scoring 85, demonstrating its effectiveness in certain query tasks. BERT + GPT-4 + GNN performs the best in terms of user satisfaction, with an overall score leading other models. T5-SQL and UnifiedSKG perform well in some user experiences, but there is still room for improvement in terms of consistency. Although PICARD is effective in some user experiences, the overall score is low, and further optimization is needed to improve overall user satisfaction.

TABLE 12 | Expanded user satisfaction scores.

Model	Original sample (10 users)	Expanded sample (50 users)	Confidence interval (95%)
BERT + GPT-4 + GNN	83.6	81.7	79.5%–83.9%
T5-SQL	79.6	78.4	76.2%–80.6%
PICARD	76.8	75.2	73.1%–77.3%
UnifiedSKG	78.5	77.6	75.4%–79.8%

4.10 | Expanded User Evaluation

To address concerns regarding the small sample size in our initial user satisfaction study, we conducted an extended evaluation involving 50 users from diverse backgrounds, including 15 data analysts, 15 business managers, and 20 nontechnical staff. The expanded study used both quantitative scoring (0–100) and qualitative feedback. The results in Table 12 show that the average satisfaction score for BERT + GPT-4 + GNN increased to 81.7, maintaining its lead over other models. T5-SQL improved to 78.4, PICARD to 75.2, and UnifiedSKG to 77.6. Qualitative feedback revealed that nontechnical users particularly appreciated the system's ability to handle complex queries without requiring SQL knowledge. However, business managers noted occasional confusion with ambiguous query results, suggesting the need for better result explanations. Data analysts highlighted the system's efficiency in generating multitable join queries but requested improvements in handling database schema evolution. The expanded evaluation strengthens our confidence in the model's practical utility while identifying specific areas for enhancement.

5 | Conclusion

This article develops a text2SQL business intelligence system based on RAG, which combines BERT, GPT-4, and GNN models to achieve efficient conversion from natural language to SQL queries, significantly improving complex query processing capabilities. The experimental results show that BERT + GPT-4 + GNN performs excellently in matching

25778196, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/eng2.70249 by Amrita Vishwa Vidyapeetham, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

accuracy of multitable joins and nested queries, with the highest user satisfaction score, verifying its superiority in practical applications. This article solves the limitations of traditional text2SQL methods in handling complex queries by utilizing the RAG framework and multimodel integration, providing enterprise users with a more convenient and efficient database query tool. The data access barriers for nontechnical personnel are reduced, and the popularization of data-driven decision-making is promoted. However, this article has limitations, including dependence on domain-specific data and potential performance bottlenecks when dealing with extremely complex nested queries. In the future, it is necessary to further optimize model parameters and training datasets, explore more efficient model integration methods, and expand the applicability of the system to cover more data types and application scenarios, so as to improve the universality and practicality of the system.

Author Contributions

Jie Liu: conceptualization, investigation, writing – original draft, software. **Shiwei Chu:** conceptualization, validation, formal analysis.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are available on request from the corresponding author. The data are not publicly available due to privacy or ethical restrictions.

References

1. G. Taher, "E-Commerce: Advantages and Limitations," *International Journal of Academic Research in Accounting Finance and Management Sciences* 11, no. 1 (2021): 153–165.

2. Z. Kedah, "Use of e-Commerce in the World of Business," *Startupreneur Business Digital (SABDA Journal)* 2, no. 1 (2023): 51–60.

3. J. Zhang, Y. Li, X. Peng, and W. Zhao, "Positive and Negative Example Induction Synthesis SQL Query Program," *Journal of Software* 34, no. 9 (2023): 4132–4152.

4. H. Cao, L. Zhao, and X. Li, "Research on Graph Neural Network Technology in Text-To-SQL Parsing," *Computer Science* 49, no. 4 (2022): 110–115.

5. G. Katsogiannis-Meimarakis and G. Koutrika, "A Survey on Deep Learning Approaches for Text-To-SQL," *VLDB Journal* 32, no. 4 (2023): 905–936.

6. Q. Li, L. Li, Q. Li, and J. Zhong, "A Comprehensive Exploration on Spider With Fuzzy Decision Text-To-SQL Model," *IEEE Transactions on Industrial Informatics* 16, no. 4 (2019): 2542–2550.

7. Y. Tian, X. Wang, Y. Wang, J. Wang, and C. Guo, "RAG-PHI: Retrieval-Augmented Generative Driven Parallel Humans and Parallel Intelligence," *Chinese Journal of Intelligent Science and Technology* 6, no. 1 (2024): 41–51.

8. D. Khurana, A. Koli, K. Khatter, and S. Singh, "Natural Language Processing: State of the Art, Current Trends and Challenges," *Multimedia Tools and Applications* 82, no. 3 (2023): 3713–3744.

9. A. Galassi, M. Lippi, and P. Torroni, "Attention in Natural Language Processing," *IEEE Transactions on Neural Networks and Learning Systems* 32, no. 10 (2020): 4291–4308.

10. A. Narechania, A. Srinivasan, and J. Stasko, "NL4DV: A Toolkit for Generating Analytic Specifications for Data Visualization From Natural Language Queries," *IEEE Transactions on Visualization and Computer Graphics* 27, no. 2 (2020): 369–379.

11. H. Kim, B.-H. So, and W.-S. Han, "Natural Language to SQL: Where Are We Today?," *Proceedings of the VLDB Endowment* 13, no. 10 (2020): 1737–1750.

12. D. Deepa, "Bidirectional Encoder Representations From Transformers (BERT) Language Model for Sentiment Analysis Task," *Turkish Journal of Computer and Mathematics Education (TURCOMAT)* 12, no. 7 (2021): 1708–1721.

13. J. He, L. Zhao, H. Yang, and M. Zhang, "HSI-BERT: Hyperspectral Image Classification Using the Bidirectional Encoder Representation From Transformers," *IEEE Transactions on Geoscience and Remote Sensing* 58, no. 1 (2019): 165–178.

14. K. M. Fitria, "Information Retrieval Performance in Text Generation Using Knowledge From Generative Pre-Trained Transformer (GPT-3)," *Jambura Journal of Mathematics* 5, no. 2 (2023): 327–338.

15. M. Zhang and J. Li, "A Commentary of GPT-3 in MIT Technology Review 2021," *Fundamental Research* 1, no. 6 (2021): 831–833.

16. C. Raffel, N. Shazeer, A. Roberts, et al., "Exploring the Limits of Transfer Learning With a Unified Text-To-Text Transformer," *Journal of Machine Learning Research* 21, no. 140 (2020): 1–67.

17. J. Zhao, M. Song, X. Gao, and Q. Zhu, "Research on Text Representation in Natural Language Processing," *Journal of Software* 33, no. 1 (2021): 102–128.

18. C. Wang, J. Ong, C. Wang, H. Ong, R. Cheng, and D. Ong, "Potential for GPT Technology to Optimize Future Clinical Decision-Making Using Retrieval-Augmented Generation," *Annals of Biomedical Engineering* 52, no. 5 (2024): 1115–1118.

19. S. Kresevic, M. Ajcevic, and A. Accardo, "Optimization of Hepatological Clinical Guidelines Interpretation by Large Language Models: A Retrieval Augmented Generation-Based Framework," *npj Digital Medicine* 7, no. 1 (2024): 102–104.

20. Y. Zhang, M. Zhu, M. Zhu, and R. Ding, "Optimizing Science Question Ranking Through Model and Retrieval-Augmented Generation," *International Journal of Computer Science and Information Technology* 1, no. 1 (2023): 124–130.

21. M. Zhao, K. Chen, L. Shou, S. Wu, and G. Chen, "Research on SQL Conversion Technology for Complex Natural Language Queries Based on Tree Model," *Journal of Software* 33, no. 12 (2022): 4727–4745.

22. P. Barcelo, E. V. Kostylev, J. L. Reutter, and P. J.-P. Silva, "The Expressive Power of Graph Neural Networks as a Query Language," *ACM SIGMOD Record* 49, no. 2 (2020): 6–17.

23. H. Li, "Codes: Towards Building Open-Source Language Models for Text-To-Sql," *Proceedings of the ACM on Management of Data* 2, no. 3 (2024): 1–28.

24. F. A. Acheampong, H. Nunoo-Mensah, and W. Chen, "Transformer Models for Text-Based Emotion Detection: A Review of BERT-Based Approaches," *Artificial Intelligence Review* 54, no. 8 (2021): 5789–5829.

25. P. Charoenkwan, C. Nantasenamat, M. Hasan, B. Manavalan, and W. Shoombuatong, "BERT4Bitter: A Bidirectional Encoder Representations From Transformers (BERT)-based Model for Improving the Prediction of Bitter Peptides," *Bioinformatics* 37, no. 17 (2021): 2556–2562.

26. A. Chan, "GPT-3 and InstructGPT: Technological Dystopianism, Utopianism, and "Contextual" Perspectives in AI Ethics and Industry," *AI and Ethics* 3, no. 1 (2023): 53–64.

27. S. Nath, A. Marie, S. Ellershaw, and P. A. Keane, "New Meaning for NLP: The Trials and Tribulations of Natural Language Processing With

25778196, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/eng2.70249 by Amrita Vishwa Vidyapeetham, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

GPT-3 in Ophthalmology," *British Journal of Ophthalmology* 106, no. 7 (2022): 889–892.

28. G. Wu, Z. Cha, L. Tu, H. Tao, and F. Song, "Research Progress in Graph Neural Network Recommendation," *CAAI Transactions on Intelligent Systems* 15, no. 1 (2020): 14–24.

29. B. Wu, X. Liang, S. Zhang, and R. Xu, "Frontiers and Applications of Graph Neural Networks," *Chinese Journal of Computers* 45, no. 1 (2022): 35–68.

30. X. Liu, Q. Shu, J. He, C. Wan, and D. Liu, "A Review of Research on Database Query Generation Based on Natural Language," *Journal of Software* 33, no. 11 (2021): 4107–4136.

31. Y. Caiyun and L. Xiaofeng, "SQL Performance Optimization Based on ORACLE Database," *Computer Knowledge and Technology* 16, no. 10 (2020): 17–19.

25778196, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/eng2.70249 by Amrita Vishwa Vidyapeetham, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License